

Glossaire — Concepts clés du cours

💡 Comment lire ce glossaire

Ce glossaire regroupe tous les concepts clés du cours, classés par thème. Chaque définition est volontairement courte — pour le détail, référez-vous à la session indiquée.

Géométrie & Pipeline (Sessions 1–2)

Vertex (sommet)

Point de base de la géométrie 3D : position (x, y, z) plus des attributs (normale, UVs, couleur). C'est la donnée d'entrée du pipeline de rendu.

Index buffer

Tableau d'indices qui réutilise les mêmes sommets pour plusieurs triangles. Évite de dupliquer les vertices partagés — économise mémoire et cache GPU.

Normale (normal)

Vecteur unitaire perpendiculaire à une surface, interpolé par vertex. Détermine comment la surface réagit à la lumière (Lambert, S5).

Coordonnées homogènes (w)

Astuce algébrique : on passe en 4D (x, y, z, w) pour que la **translation** devienne une multiplication de matrices comme les autres transformations. $w = 1$ point, $w = 0$ direction.

Matrice de transformation

Matrice 4×4 qui combine scale, rotation et translation. Une seule multiplication transforme des milliers de sommets d'un coup.

Ordre des transformations

Les matrices se multiplient de **droite à gauche** : $M = T \cdot R \cdot S$ applique S d'abord, puis R, puis T. Non commutatif — l'ordre change le résultat.

Espaces de coordonnées

Local (repère de l'objet) $\rightarrow$ **Monde** (scène) $\rightarrow$ **Caméra** (view) $\rightarrow$ **Clip** (projection) $\rightarrow$ **Écran**.
Chaque espace a sa matrice de passage.

Matrice World / View / Projection

World : local $\rightarrow$ monde. **View** : monde $\rightarrow$ caméra (inverse de la caméra). **Projection** : caméra $\rightarrow$ clip space (perspective ou orthographique).

Parenting (hiérarchie de scène)

Un enfant hérite des transformations de son parent : sa matrice monde est $W_{\text{enfant}} = W_{\text{parent}} \cdot L_{\text{enfant}}$. Le train et le passager.

Frustum

Volume de vision de la caméra en forme de pyramide tronquée (near/far planes). Seule la géométrie à l'intérieur est rendue.

Shaders & GLSL (Sessions 3–4)

Shader

Programme exécuté sur le GPU. Contrairement au CPU (quelques cœurs rapides), le GPU a des milliers de petits cœurs — idéal pour traiter des millions de pixels en parallèle.

Vertex shader

Première étape programmable : transforme chaque sommet (matrices MVP) et calcule les **varyings** transmis au fragment shader.

Fragment shader

Dernière étape programmable : calcule la **couleur** de chaque pixel. C'est là que vivent la lumière, les textures et les effets.

Rasterizer

Étape fixe entre vertex et fragment shader : détermine quels pixels sont couverts par chaque triangle, et **interpole** les varyings.

Uniform

Variable globale passée du CPU au shader, **identique** pour tous les sommets/pixels d'un draw call : time, worldViewProjection, lightPos.

Varying

Variable interpolée par le rasterizer entre les sommets d'un triangle et lue par le fragment shader : vNormal, vUV, vPosition.

Attribute

Donnée par sommet en entrée du vertex shader : position, normal, uv. Lue depuis le vertex buffer.

GLSL

Langage des shaders : typé (vec3, mat4), fonctions intégrées (dot, normalize, mix, step, smoothstep, clamp), swizzling (v.xzy).

step()

step(edge, x) retourne 0 si $x < \text{edge}$, 1 sinon. Crée une transition **dure** — seuil, bande, masque binaire.

smoothstep()

Comme step mais transition **douce** (hermite) entre deux bornes. L'outil de base pour des dégradés propres et des bords adoucis.

mix()

Interpolation linéaire : $\text{mix}(a, b, t) = a(1 - t) + bt$. L'équivalent GLSL de lerp — mélange couleurs, positions, valeurs.

ShaderMaterial (Babylon.js)

Matériau Babylon qui accepte du code GLSL brut : on fournit vertex + fragment shader, plus les uniforms et les attributs.

Lumière & Matériaux (Session 5)

Loi de Lambert

L'éclairement d'une surface est proportionnel à $\cos(\theta) = \vec{N} \cdot \vec{L}$. Perpendiculaire à la lumière → plein éclairement ; rasante → faible ; dos tourné → nul.

Diffus

Composante de lumière réémise uniformément dans toutes les directions — la “couleur de base” de la surface. Dépend de $\vec{N} \cdot \vec{L}$, pas de la caméra.

Spéculaire

Reflét **dépendant de la vue** : la tache brillante sur une surface lisse. Dépend de l'angle entre le reflet et l'œil — c'est ce qui déplace le reflet quand la caméra bouge.

Modèle de Phong

Specular = $\max(0, \vec{R} \cdot \vec{V})^{\alpha}$ où $\vec{R}$ est la lumière réfléchie et α le **shininess** — grand = tache petite et dure.

Blinn-Phong

Variante optimisée : on utilise le **half-vector** $\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|}$ et $\max(0, \vec{N} \cdot \vec{H})^{\alpha}$ — pas besoin de calculer $\vec{R}$.

Ambient

Terme constant ajouté partout pour simuler la lumière indirecte — évite les ombres complètement noires. Approximation, pas une vraie physique.

DirectionalLight

Lumière à l'infini définie par une **direction** seule — le soleil. Tous les rayons sont parallèles.

PointLight / OmniLight

Lumière ponctuelle qui rayonne dans toutes les directions depuis une position, avec **atténuation** selon la distance. Ampoule, torche.

SpotLight

PointLight restreint à un **cône** : direction + angle + pénombre douce au bord du cône.

HemisphericLight

Ambiance améliorée : deux couleurs (ciel/sol) interpolées selon la normale. Simule le ciel bleu qui éclaire par le haut et le sol qui renvoie par le bas.

Matériau

Ensemble de paramètres qui décrivent la réponse d'une surface à la lumière : diffuseColor, specularColor, shininess, textures, shader.

Éclairage Statique (Session 6)

Baking (cuisson)

Pré-calculer hors ligne un résultat coûteux (éclairage, ombres, AO) dans une texture, puis le **lire** en temps réel. Échange mémoire contre performance.

Lightmap

Texture contenant l'éclairage pré-calculé d'une surface : le shader fait juste `texture(lightmap, uv2) * couleur`. Quasi gratuit à l'exécution.

UV2

Deuxième canal UV dédié aux lightmaps : un dépliage **sans chevauchement** (chaque texel = une zone unique de la surface). Distinct des UVs de texture.

Ambient Occlusion (AO)

Assombrissement des creux et recoins où la lumière ambiante a du mal à pénétrer. Donne le contact visuel, la “saleté” des coins — change tout visuellement.

Baked AO vs SSAO

Baked : pré-calculé dans une texture, précis, gratuit, mais statique. **SSAO** : estimé à l’écran à l’exécution, dynamique, mais approximatif et coûteux.

Light Probe

Point de capture qui stocke la lumière ambiante d’un lieu. Les objets **dynamiques** échantillonnent les probes pour recevoir l’éclairage baked.

Spherical Harmonics (SH)

Compression de la lumière ambiante en quelques coefficients (souvent 9). Stocke un champ lumineux directionnel quasi gratuit à évaluer — le format des light probes.

Textures (Session 7)

Texture

Image appliquée sur la géométrie via les UVs : couleur, relief, rugosité. Le principal levier de richesse visuelle sans géométrie supplémentaire.

UV mapping

Correspondance 2D→3D : chaque sommet porte des coordonnées $(u, v) \in [0, 1]^2$ qui indiquent quel texel de la texture lui correspond. Le “dépliage” du modèle.

Seam (couture)

Arête où le dépliage est coupé : les UVs discontinus créent une ligne visible. On place les seams là où l’œil ne les verra pas.

UV island

Morceau connexe du dépliage. Plus d’îlots = plus de coutures, mais moins d’étirement. Compromis fondamental du dépliage.

Texel

“Pixel de texture”. Quand un texel couvre plus d’un pixel d’écran → magnification ; moins d’un → minification (le cas difficile).

Nearest vs Bilinear

Nearest : prend le texel le plus proche → pixelisé. **Bilinear** : moyenne des 4 texels voisins → lisse. Filtrage de **magnification**.

Mipmap

Pyramide de versions réduites de la texture (1/2, 1/4, 1/8...). Le GPU choisit le niveau adapté à la distance — résout la minification et le shimmering.

Trilinear / Anisotrope

Trilinear : bilinear dans deux mipmaps + mix entre niveaux. **Anisotrope** : échantillonnage directionnel — indispensable pour les sols en perspective rasante.

Wrapping

Comportement aux UVs > 1 ou < 0 : **Repeat** (tuile), **Clamp** (bord étiré), **Mirror** (tuile en miroir).

sRGB vs Linéaire

Les textures couleur sont stockées en sRGB (perceptuel), le shader calcule en **linéaire**. Oublier la conversion → couleurs délavées ou trop sombres.

Textures Avancées & Baking (Sessions 8–9)

Set PBR

Collection de maps qui décrivent un matériau physique : **albedo** (couleur), **normal** (relief), **roughness** (rugosité), **metallic** (métal), **AO**.

Normal map

Texture qui encode des **normales perturbées** par texel (RGB = XYZ en espace tangent). Donne du relief visuel **sans géométrie** — la brique de base du détail.

Espace tangent

Repère local à chaque texel (tangent, bitangent, normale) dans lequel la normal map est écrite. Le shader y transforme la lumière et la vue.

Height map / Displacement

Carte d'élévation : peut déformer la géométrie (displacement, coûteux) ou juste parallaxer la texture (parallax mapping, illusion).

Texture procédurale

Image générée par du **code** plutôt que stockée : bruit, motifs, marbre, bois. Résolution infinie, zéro mémoire, paramétrable.

Hash

Fonction déterministe $f(x) \rightarrow$ aléatoire **stable** : le même x donne toujours la même valeur. Le cœur de tout bruit procédural (ex. `fract(sin(x) * 43758)`).

Value noise

Bruit le plus simple : valeurs aléatoires aux coins d'une grille, interpolées. Rapide mais montre la grille en diagonale.

Perlin noise

Bruit **gradient** : vecteurs aléatoires aux coins, on interpole les produits scalaires. Plus organique, le standard historique.

Simplex noise

Évolution de Perlin : grille **simplex** (triangles/tétraèdres) — moins d'artefacts directionnels, plus rapide en 3D+.

Worley noise (cellular)

Bruit cellulaire : distance au point aléatoire le plus proche. Produits cellules, pierres, écailles, crevasses.

FBM (Fractal Brownian Motion)

Somme de plusieurs octaves de bruit à fréquence doublée / amplitude moindre : $f(x) + \frac{1}{2}f(2x) + \frac{1}{4}f(4x) + \dots$. Le détail à toutes les échelles — nuages, montagnes, marbre.

Octave

Une couche de bruit dans le FBM : chaque octave double la fréquence et réduit l'amplitude. 4–8 octaves suffisent.

Bake (Blender/Cycles)

Transférer une propriété (diffuse, normal, AO) d'un modèle vers une texture via le moteur Cycles : high-poly $\rightarrow$ low-poly, ou matériau $\rightarrow$ texture.

Cage

Enveloppe utilisée lors du bake de normal map : projette les rayons depuis la low-poly vers la high-poly. Mal réglée = artefacts.

Ombres (Session 10)

Shadow mapping

Technique d'ombre en deux passes : (1) rendre la profondeur **vue par la lumière** dans une texture, (2) pour chaque fragment, comparer sa profondeur à celle de la map — plus loin = à l'ombre.

Shadow map

Texture de **profondeur** rendue depuis le point de vue de la lumière. Pas de couleur — juste “à quelle distance la lumière voit-elle la première surface ?”

Coordonnées d'ombre

Transformer le fragment dans l'espace **clip de la lumière** pour lire la shadow map au bon texel : matrice light view-projection + bias.

Shadow acne

Artefact de bandes moirées : la profondeur quantifiée de la map se bat contre la surface elle-même. Se règle par un **bias** (décalage de profondeur).

Peter Panning

L'inverse de l'acne : trop de bias détache l'ombre de son objet (l'ombre “flotte”). Le bias est un compromis entre les deux.

PCF (Percentage Closer Filtering)

Adoucissement des ombres : échantillonner plusieurs texels voisins de la shadow map et **moyenner** le test → bords d'ombre progressifs.

Cascade Shadow Map (CSM/PSSM)

Plusieurs shadow maps couvrant des tranches de distance croissante : résolution fine près, grossière loin. Indispensable pour les grandes scènes directionnelles.

Ombre dure vs douce

Dure : test binaire, bord net (coût minime). **Douce** : pénombre via PCF ou VSM — plus réaliste, plus coûteuse.

Post-Processing (Session 11)

Render target

Texture dans laquelle on rend la scène au lieu de l'écran. Permet de relire l'image pour la retraiter — la base de tout post-processing.

Fullscreen quad

Un simple rectangle couvrant l'écran sur lequel on applique le shader d'effet : l'image devient une **texture** à manipuler comme n'importe quelle autre.

Ping-pong rendering

Alterner entre deux render targets ($A \rightarrow B \rightarrow A$) pour enchaîner les passes d'effets sans qu'un shader lise et écrive la même texture.

Tone mapping

Compression de la plage HDR (valeurs > 1) vers le LDR de l'écran : Reinhard, ACES, Filmic. Préserve les détails des hautes lumières.

Gamma correction

Conversion linéaire $\rightarrow$ sRGB en fin de chaîne : l'écran et l'œil sont non-linéaires, le calcul se fait en linéaire.

Bloom

Halo lumineux autour des zones brillantes : extraire les pixels $>$ seuil $\rightarrow$ flou gaussien $\rightarrow$ addition à l'image. Simule le débordement de lumière.

Depth of Field (DoF)

Flou selon la distance à la **focale** : avant-plan et arrière-plan flous, sujet net. Utilise le depth buffer comme carte de flou.

SSAO

Ambient occlusion estimée à **l'écran** en sondant le depth buffer autour de chaque pixel. Dynamique et approximatif — voir AO baked (S6).

FXAA / MSAA / TAA

Anticrénelage : **MSAA** échantillonne plus par pixel (géométrie), **FXAA** floute les bords en post (gratuit), **TAA** accumule temporellement (net et dynamique).

Optimisation & Grandes Scènes (Session 12)

Draw call

Un appel de rendu CPU→GPU. Chaque draw call a un coût fixe de préparation — des milliers d'objets = goulot CPU avant même le GPU.

Frustum culling

Ne pas envoyer au GPU les objets **hors du frustum** de la caméra. Test trivial par bounding box — l'économie la plus rentable.

Backface culling

Ne pas rasteriser les triangles dont la normale **fuit** la caméra. Moitié des triangles d'un objet fermé éliminés gratuitement.

Occlusion culling

Ne pas dessiner les objets **cachés** par d'autres — le plus difficile à prédire. PVS, portails, ou requêtes GPU.

Structure spatiale

Organisation hiérarchique de la scène pour trouver vite ce qui est visible : **Octree** (8 sous-boîtes), **Quadtrees** (2D), **BVH** (boîtes englobantes emboîtées), **Grid**.

BVH (Bounding Volume Hierarchy)

Arbre de boîtes englobantes emboîtées : tester une boîte élimine d'un coup tout son sous-arbre. Structure dominante du culling et du raytracing.

LOD (Level of Detail)

Plusieurs versions d'un même objet (haute/moyenne/basse résolution) ; le moteur choisit selon la distance. Économise vertices et fill-rate.

Popping / transition LOD

Saut visuel quand le LOD change. Adouci par **dithering** (mélange temporel des deux LOD) ou choix de seuils avec hystérésis.

Billboard

Quad orienté face caméra avec une texture : simule un objet 3D complexe (arbre, personnage lointain) à 2 triangles. Le LOD extrême.

Impostor

Billboard évolué : texture rendue **depuis le bon angle** (pré-calculée ou dynamique) pour imiter la parallaxe d'un vrai objet.

Instancing

Dessiner N copies du même mesh en **un** draw call avec un tableau de transforms. Forêts, foules, particules — l'arme anti-draw-calls.

Fill-rate / overdraw

Coût par **pixel** dessiné : les surfaces transparentes qui se superposent (particules, verre) écrivent le même pixel plusieurs fois.

💡 Le fil conducteur

Le cours entier est un même compromis : **qualité vs coût**. Lightmaps et baking échangent mémoire contre calcul ; LOD et billboards échangent fidélité contre vitesse ; les shaders décident ce qui vaut la peine d'être calculé **par pixel** vs pré-calculé. Retenez cette question pour chaque technique : **quel est mon budget temps de calcul, et de combien de mémoire je dispose ?**